CF Bulk Upload – Delete Challenges

Adding DELETE to the CF bulk uploader is a small code change, but it is the only action that cannot be undone by uploading again — so it needed a decision on safeguards before we built it.

Status: decided and built (20 Oct 2026)

After the demo, DELETE was agreed and built as a real delete with these safeguards:

#	Decision	How it works
1	Real delete, no archive folder	The fragment is removed. Save a report before a big change to keep its values
2	Referenced by other fragments: not deleted	`WARN Not deleted: referenced by … (remove the references first)`, listing up to five fragments under the picked folder
3	Sites pages and experience fragments are checked	AEM's reference search; they are listed in the same warning
4	Published: not deleted	`WARN Not deleted: still published (unpublish it first)`
5	Safeguard: dry run first	Final run is enabled only after a dry run with no failures; no extra checkbox or OSGi switch
6	Who may delete	AEM permissions of the author running the upload; no extra group
8	Order	Deletes run after all CREATE / UPDATE rows, sheets in reverse profile order (offers first, venues last); a fragment deleted earlier in the run no longer counts as a reference
9	Audit trail	Each final run saves a results report in `/content/dam/aemcloudproject/reports`
10	Language copies	Out of scope
11	Speed	References inside the picked folder are read once per upload

Still open:

- **Reused IDs (7):** new IDs are now the highest in the sheet **or in DAM** + 1. Deleting the fragment with the highest ID still lets the next CREATE reuse its ID.

The proposal and challenges below are kept as written before the decision.

Proposal

Today the uploader supports CREATE and UPDATE; deletes are done by hand in AEM. The proposal adds `DELETE` as a third value in the action column.

action	slug	other columns
DELETE	summer-sale	ignored

- Only the slug column is needed; required fields, ID generation and value checks don't apply.
- A dry run shows what would be deleted, without deleting.
- The result line reads `DELETE PASS …/offers/summer-sale`, or FAIL with the reason.
- Before deleting, the tool would check: the fragment exists, it belongs to this sheet's model, nothing references it, and it isn't published.

Challenges

The biggest risks are irreversibility, broken references and fragments that are still live on publish.

#	Challenge	Why it matters	Options
1	Can't be undone	DAM has no recycle bin we can rely on; recovery means a backup restore. Every other action can be fixed by uploading again.	Real delete, or move to an archive folder that can be restored by moving back
2	Broken references between fragments	Merchants reference venues; offers reference merchants, categories, cards and CTAs. Deleting a referenced fragment leaves dead paths behind.	Block the delete and list who references it, or allow it with a warning
3	Usage outside DAM	Sites pages, GraphQL queries and headless apps may use the fragment. The uploader can't see them, so the check in 2 doesn't cover them.	Also search Sites pages, or accept the risk and document it
4	Fragment is published	Deleting on author leaves the copy on publish and the CDN; frontends keep showing it.	Block until unpublished, or unpublish automatically before deleting
5	Accidental deletes	Excel makes mistakes easy: fill-down of the action column, a copied row, the wrong sheet. One wrong cell deletes live content.	An "Allow deletes" checkbox (off by default), an OSGi on/off switch per environment, or both
6	Who may delete	The upload runs with the author's own rights; today every author with write access could bulk-delete.	Rely on AEM permissions, or limit DELETE to an admin group
7	Reused IDs	New IDs are "highest in sheet + 1". If the fragment with the highest ID is deleted, the next CREATE gets that same ID; any system that stored it now points at a different offer.	Never reuse: track the highest ID ever issued, or take the highest from AEM plus the archive
8	Order inside one workbook	A row can UPDATE an offer to reference a CTA that another row deletes in the same upload.	Process deletes last, and check references against the rows of the same upload
9	Audit trail	Only the results CSV and server logs show who deleted what. Nothing is kept in AEM.	Keep the archive (covers it), or write a small delete log under /var
10	Language copies and variations	Deleting a fragment removes its variations; language copies in other locales are separate fragments and would remain.	Out of scope, or delete per language explicitly
11	Speed	The reference check reads every fragment once per DELETE row. Many deletes in one upload can hit the request timeout on Cloud.	Read references once per upload, and keep deletes within the row limit

Decisions needed

We needed these answers before building DELETE; each maps to a row in the table above. See the status table at the top for the answers.

☑ Real delete or archive folder? (1) — real delete

☑ Referenced by other fragments: block or allow with a warning? (2) — not deleted, with a warning

☑ Check Sites pages and other consumers, or accept the risk? (3) — Sites pages and experience fragments are checked

☑ Published fragments: block, or unpublish automatically? (4) — not deleted, with a warning

☑ Safeguard: "Allow deletes" checkbox, OSGi switch, both, or none? (5) — the dry run / final run gate

☑ Who may delete: all authors with write access, or an admin group only? (6) — AEM permissions

☐ Must deleted IDs never be reused? (7) — open

☑ Language copies: in scope or not? (10) — out of scope

Our suggestion

Start with the safest version: archive instead of delete, and refuse anything risky.

- **Archive, don't delete:** move to `cfs/_deleted/<date>/…`; it can be restored by moving it back, and the archive doubles as the audit trail.
- **Refuse referenced or published fragments,** with a clear reason in the results; the author fixes those by hand first.
- **"Allow deletes" checkbox,** off by default; a dry run always shows what would be deleted.
- **Never reuse IDs:** generated IDs also look at archived fragments.
- **Deletes run last** in an upload, after all CREATE and UPDATE rows.

If this version works well, switching archive to a real delete later is a small change.